pfSense + Kali + Ubuntu DoS Defence Lab

Objective

Stand up a three-VM home lab demonstrating a basic firewall defence scenario:

- A **pfSense** firewall separating an external ("WAN") network from an isolated internal ("LAN") network.
- A **Kali Linux** VM on the WAN side acting as an external attacker.
- An **Ubuntu Desktop** VM on the LAN side acting as the victim.
- Launch a denial-of-service flood (ICMP/SYN via `hping3`) from Kali against Ubuntu, observe it in Wireshark, then write a pfSense firewall rule to block it and verify the block in the logs.

Final Topology & IP Plan

Device / Segment	Adapter	Interface	IP / Subnet	Role
Home network (physical)	— (host network)	—	< — home network range>	Provides internet, bridges to the lab
pfSense WAN	Bridged	em0	<-WAN-IP> (DHCP from home router)	Edge of the firewall toward the home network
Kali Linux (Attacker)	Bridged	eth0	<-Kali-IP> (DHCP from home router)	External attacker host
Lab LAN (internal)	VMnet1 (Host-only)	—	192.168.1.0/24	Isolated subnet behind pfSense
pfSense LAN	VMnet1 (Host-only)	em1	192.168.1.1 (DHCP range .100–.199)	Default gateway & DHCP server for the LAN
Ubuntu Desktop (Victim)	VMnet1 (Host-only)	ens33	192.168.1.100 (DHCP from pfSense)	Victim workload

Building the VMs in VMware Workstation

pfSense

- Guest OS: Other → FreeBSD 64-bit (VMware could not auto-detect the ISO's OS)
- 2 vCPU, 2048 MB RAM, 20 GB disk (single file)
- Network Adapter 1: **Bridged** (WAN)
- Network Adapter 2: **Custom: VMnet1 (Host-only)** (LAN)
- Installed pfSense accepting defaults; assigned **em0 = WAN**, **em1 = LAN** at the console interface-assignment step (VMware's e1000 driver presents interfaces as `em0`/`em1` rather than VirtualBox's `vtnet0`/`vtnet1`)
- WAN set to DHCP (client), no VLANs

Kali Linux (Attacker)

- Guest OS: Linux → Debian 64-bit (closest match; Kali itself isn't a listed preset)
- 4 vCPU, 4096 MB RAM, 20 GB disk
- Network Adapter: **Bridged**
- Installed via **Graphical Install** (manual, not VMware's "Easy Install" automation), hostname `kali`, no domain, Guided partitioning — entire disk, single partition, GRUB installed to `/dev/sda`
- Software selection: default Xfce desktop + default tools metapackage (includes `hping3`)
- Installed cleanly on the first attempt in VMware with no hangs, supporting the conclusion that the earlier VirtualBox hangs were hypervisor-related

Ubuntu Desktop (Victim)

- Guest OS: Linux → Ubuntu 64-bit
- 2 vCPU (later effectively unconstrained — see below), 20 GB disk
- Network Adapter: **Custom: VMnet1 (Host-only)** — same VMnet as pfSense's LAN adapter, so Ubuntu and pfSense's LAN side share a segment
- **Caught a download mistake mid-setup:** the first Ubuntu ISO downloaded was the **ARM64** build, which VMware refused to power on ("requires the Arm machine architecture... incompatible with this x86 machine architecture host"). Re-downloaded the correct **AMD64 (x86_64)** Desktop ISO before proceeding.
- Install type: Interactive installation, Default app selection, no proprietary drivers (not needed for VMware's standard virtual hardware), Erase disk and install Ubuntu, no disk encryption
- After installation completed, the VM kept returning to a live/installer-style environment on reboot. Found the cause: **two CD/DVD devices were still attached** — the real Ubuntu ISO plus an `autoinst.iso` VMware auto-generates as its "Easy Install" helper disc. Detached **both**, which allowed it to boot from the installed virtual disk correctly.

Networking Troubleshooting

Two separate connectivity problems came up once all three VMs were installed, each with a distinct root cause:

1. Ubuntu had no internet access

Ubuntu's `ip a` showed an address in the `192.168.114.x` / `192.168.115.x` range — not pfSense's `192.168.1.0/24` LAN subnet. The cause: **VMware's Virtual Network Editor runs its own built-in DHCP server on host-only networks (VMnet1) by default**, and it was winning the DHCP race against pfSense, handing Ubuntu an address before pfSense's DHCP server had a chance.

Fix: Edit → Virtual Network Editor → select VMnet1 → uncheck "Use local DHCP service to distribute IP address to VMs." After disabling VMware's DHCP and rebooting both VMs (pfSense first, then Ubuntu), Ubuntu correctly picked up `192.168.1.100` from pfSense.

2. Kali couldn't reach Ubuntu even with a pass rule in place

The actual cause, confirmed by testing that pfSense's own console *could* ping Ubuntu directly (ruling out the LAN/VMnet1 segment) while Kali could not reach it at all: **Kali had no route to the `192.168.1.0/24` subnet.** Kali's default gateway is the home router (since it's bridged onto the physical network), and the home router has no knowledge of the lab's internal subnet — so packets destined for Ubuntu were being sent nowhere. This matches the guide's own Section 6 instruction, which was missed in the initial pass:

```
None
sudo ip route add 192.168.1.0/24 via <PFSENSE_WAN_IP>
```

(using pfSense's current WAN IP, `<redacted-WAN-IP>`). This route is **not persistent** — it must be re-added after every Kali reboot unless scripted. Once added, `pinging 192.168.1.100` from Kali succeeded, confirming the full path: Kali → pfSense WAN → routed to LAN → Ubuntu.

Firewall Rules Reference (pfSense → WAN tab)

Order	Action	Protocol	Source	Destination	Purpose
—	Pass	TCP:443	Home network	pfSense WAN	Allow GUI access from the home network
—	Pass	ICMP (later widened to Any)	Kali IP (<redacted-Kali-IP>)	Ubuntu IP (192.168.1.100)	Allow Kali to reach Ubuntu for the demo

Added later, at top	Block (logged)	Any	Kali IP	Ubuntu IP	Block the simulated attack

Lesson: the pass rule initially only allowed **ICMP**, which is enough for `ping` but blocks a TCP-based SYN flood test. Widening the rule's Protocol to **Any** (or adding a second TCP rule) was necessary before a SYN flood (`hping3 -S -p 80`) would reach Ubuntu at all.

Running the DoS Demo

- Installed Wireshark on Ubuntu: `sudo apt update && sudo apt install -y wireshark` (had to enable the **universe** repository first via `sudo add-apt-repository universe` when `apt` reported it couldn't locate the package); allowed non-superusers to capture packets (adds the user to the `wireshark` group) rather than routinely running the GUI as root.
- Captured on interface `ens33`, filtered on `icmp`.
- For a safer, still-clearly-visible demonstration, used a **rate-limited** flood instead:

```
None
  sudo hping3 -1 -i u10000 192.168.1.100
```

(one ICMP echo request every 10 ms). This produced a clean, steady stream of Echo Request/Reply pairs in Wireshark on Ubuntu — clear evidence of the "before" (unblocked) state — without disrupting the home network.

Blocking the Attack

- Added a new rule at the **top** of Firewall → Rules → WAN: Action **Block**, Protocol **Any**, Source = Kali IP, Destination = Ubuntu IP, **Log** enabled, Description "Block Kali DoS."
- Rule order matters in pfSense — it evaluates top-to-bottom and stops at the first match, so the block rule must sit above the earlier pass rule to take effect.
- After applying, re-running the same `hping3` command from Kali produced no replies, and Wireshark on Ubuntu showed no new ICMP traffic arriving — confirming the block.
- pfSense's **Status** → **System Logs** → **Firewall** shows the blocked packets being logged, giving a clean before/after artefact of the mitigation.